

就可以按照统一的方式访问不同生产厂商的设备了。应用程序使用某个具体设备之前只需安装该设备的驱动程序,并按照标准的接口方式使用即可,从而最大限度地规范了应用程序的开发。具体到数据库的场景,就是**开放数据库互连**(open database connectivity, ODBC)。

ODBC 是微软公司 WOSA 中有关数据库的一个组件,它建立了一组规范,并提供一组数据库应用编程接口(database application programming interface, DBAPI)。这样,无论使用什么数据库,都采用同样的一组编程接口来访问数据库,规范了应用程序的开发,提高了应用程序的可移植性。

数据库系统需要各自实现这组编程接口,形成该数据库的驱动程序。利用 ODBC 访问数据库之前需要安装数据库驱动程序, WOSA 中有一个数据库驱动程序管理器,负责管理各种数据库的驱动程序。

利用 ODBC 访问数据库的流程如下:

- ① 环境准备。配置数据源,即某个数据库,包含数据库名称,访问地址,用户名,口令等信息。需要用驱动程序管理器工具来完成。
- ② 设置环境。利用编程接口中的环境设置语句进行设置。
- ③ 进行数据库连接。
- ④ 完成相关操作。执行 SQL 语句,完成对数据库的相关操作、对返回结果集进行计算等。
- ⑤ 关闭数据库连接,释放占用的资源。

类似地,在微软之外的平台上,也有其他的一些机制,例如 JDBC(适用于 Java 环境)等。由于 Java 语言使用的普遍性,本书将在 8.3 节详细介绍如何使用 JDBC 访问数据库中的数据。

软件的体系结构在不断发展变化,访问数据库的方式也会随之发生变化,该过程不会终止。期待有更便捷高效的应用程序与数据库之间的交互方式的出现。

8.2 过程化 SQL

SQL 99 标准支持存储过程和函数的概念,商用关系数据库管理系统大多提供过程化的 SQL,如 Oracle 的 PL/SQL、Microsoft SQL Server 的 Transact-SQL、IBM DB2 的 SQL PL、Kingbase 的 PL/SQL 等。本节主要介绍过程化 SQL 的块结构、变量和常量的定义、流程控制、游标的定义与使用、存储过程及存储函数等内容。

8.2.1 过程化 SQL 的块结构

基本 SQL 是高度非过程化的语言,而过程化 SQL 是对基本 SQL 的扩展,在其基础上增加了一些描述过程控制的语句。

过程化 SQL 程序的基本结构是块,每个块可以包含定义部分和执行部分,块与块之间可以相互嵌套,每个块完成一个逻辑操作。图 8.4 是过程化 SQL 块的基本结构。

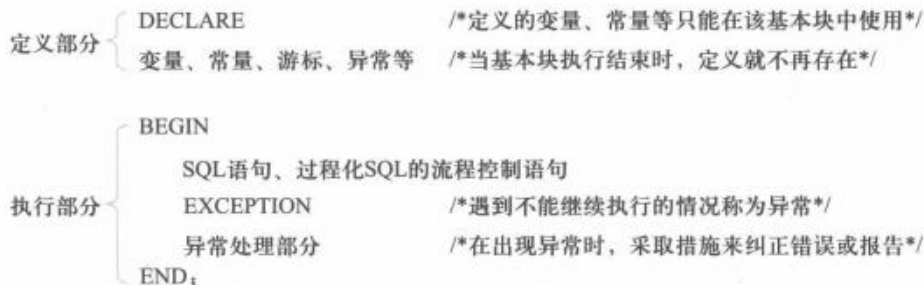


图 8.4 过程化 SQL 块的基本结构

8.2.2 变量和常量的定义

变量的定义格式如下：

变量名 数据类型 [[NOT NULL] := 初值表达式]

或

变量名 数据类型 [[NOT NULL] 初值表达式]

常量的定义格式如下：

常量名 数据类型 CONSTANT := 常量表达式

常量必须要赋予一个值，并且该值在存在期间或常量的作用域内不能改变。如果试图修改它，过程化 SQL 将返回一个异常。

赋值语句的格式如下：

变量名 := 表达式

8.2.3 流程控制

过程化 SQL 提供了流程控制语句，主要有条件控制语句和循环控制语句。这些语句的语法、语义和一般的高级语言(如 C 语言)类似，这里只做概要介绍。读者使用时需参考具体产品手册的语法规则。

1. 条件控制语句

条件控制语句一般包含三种形式的 IF 语句：IF-THEN 语句、IF-THEN-ELSE 语句和嵌套的 IF 语句。

① IF-THEN 语句的格式如下:

```
IF condition THEN
    Sequence_of_statements;    /* 条件为真时语句序列才被执行 */
END IF; /* 条件为假或 NULL 时什么也不做,控制转移至下一个语句 */
```

② IF-THEN-ELSE 语句的格式如下:

```
IF condition THEN
    Sequence_of_statements1;    /* 条件为真时执行语句序列 1 */
ELSE
    Sequence_of_statements2;    /* 条件为假或 NULL 时执行语句序列 2 */
END IF;
```

③ 嵌套的 IF 语句。在 THEN 和 ELSE 子句中还可以再包含 IF 语句,即 IF 语句可以嵌套。

2. 循环控制语句

过程化 SQL 有三种循环控制语句: LOOP 语句、WHILE-LOOP 语句和 FOR-LOOP 语句。

① 最简单的循环语句 LOOP 格式如下:

```
LOOP
    Sequence_of_statements;    /* 循环体,一组过程化 SQL 语句 */
END LOOP;
```

多数数据库服务器的过程化 SQL 都提供 EXIT、BREAK 或 LEAVE 等循环结束语句,以保证 LOOP 语句块能够在适当的条件下提前结束。

② WHILE-LOOP 循环语句的格式如下:

```
WHILE condition LOOP
    Sequence_of_statements;    /* 条件为真时执行循环体内的语句序列 */
END LOOP;
```

每次执行循环体语句之前首先要对条件进行求值,如果条件为真则执行循环体内的语句序列,如果条件为假则跳过循环并把控制传递给下一个语句。

③ FOR-LOOP 循环语句的格式如下:

```
FOR count IN [ REVERSE ] bound1 .. bound2 LOOP
    Sequence_of_statements;
END LOOP;
```

FOR 循环的基本执行过程是:将 count 设置为循环的下界 bound1,检查它是否小于上界 bound2。当指定 REVERSE 时则将 count 设置为循环的上界 bound2,检查 count 是否大于下界 bound1。如果越界则执行跳出循环,否则执行循环体,然后按照步长(+1 或-1)更新 count 的值,重新判断条件。

3. 错误处理

如果过程化 SQL 在执行时出现异常, 则应该让程序在产生异常的语句处停下来, 根据异常的类型执行异常处理语句。

SQL 标准对数据库服务器提供什么样的异常处理给出了建议, 要求过程化 SQL 管理器提供完善的异常处理机制。相对于嵌入式 SQL 简单地提供执行状态信息 SQLCODE, 这里的异常处理就复杂多了。读者要根据具体系统的支持情况进行错误处理。

8.2.4 游标的定义与使用

在过程化 SQL 中, 如果 SELECT 语句只返回一条记录, 可以将该结果存放到变量中。当查询返回多条记录时, 就要使用游标对结果集进行处理。一个游标与一个 SQL 语句相关联。

使用游标的步骤如下。

1. 声明游标

用 DECLARE 语句为一条 SELECT 语句定义游标:

```
DECLARE 游标名 [(参数1 数据类型, 参数2 数据类型, ...)] /* 不同的关系数据库产品语法稍有不同 */  
CURSOR FOR /* 在 Kingbase 中是"游标名 CURSOR FOR" */  
SELECT 语句; /* 在 Oracle 中是"CURSOR FOR 游标名" */
```

定义游标仅仅是一条说明性语句, 这时关系数据库管理系统并不执行 SELECT 语句。

2. 打开游标

用 OPEN 语句将定义的游标打开:

```
OPEN 游标名 [(参数1 数据类型, 参数2 数据类型, ...)];
```

打开游标实际上是执行相应的 SELECT 语句, 把查询结果取到缓冲区中。这时游标处于活动状态, 指针指向查询结果集中的第一条记录。

3. 使用游标

推进游标指针并取当前记录:

```
FETCH 游标名 INTO 变量1[, 变量2, ...];
```

其中变量必须与 SELECT 语句中的目标列表表达式具有一一对应关系。

用 FETCH 语句把游标指针向前推进一条记录, 同时将缓冲区中的当前记录取出送至变量供过程化 SQL 进一步处理。通过循环执行 FETCH 语句逐条取出结果集中的行进行处理。

4. 关闭游标

用 CLOSE 语句关闭游标, 释放结果集占用的缓冲区及其他资源。

```
CLOSE 游标名;
```

游标被关闭后就不再和原来的查询结果集相联系,但被关闭的游标可以再次被打开,与新的查询结果相联系。

[例 8.4] 根据给定学号 20180001, 使用游标输出该学生的全部选课记录。

```
DECLARE
    CnoOfStudent CHAR(10);
    GradeOfStudent INT;
    mycursor CURSOR FOR
    SELECT Cno, Grade FROM SC WHERE Sno='20180001';
BEGIN
    OPEN mycursor;    /* 打开游标 */
    LOOP              /* 循环遍历游标 */
        FETCH mycursor INTO CnoOfStudent, GradeOfStudent; /* 检索游标 */
        EXIT WHEN mycursor%NOTFOUND;
        RAISE NOTICE 'Sno:20180001, Cno:%, Grade:%', CnoOfStudent, GradeOfStudent;
    END LOOP;
    CLOSE mycursor;   /* 关闭游标 */
END;
```

8.2.5 存储过程

类似于高级语言程序,过程化 SQL 程序也可以被命名和编译并保存在数据库中,称为存储过程(stored procedure)或存储函数(stored function),供其他过程化 SQL 调用。存储过程或存储函数也是一类数据库的对象,需要有创建、删除等语句。

1. 创建存储过程

```
CREATE OR REPLACE PROCEDURE 过程名(
    [[IN|OUT|INOUT] 参数1 数据类型,
    [IN|OUT|INOUT] 参数2 数据类型,...]
) /* 存储过程首部 */
AS <过程化 SQL 块>; /* 存储过程体,描述该存储过程的操作 */
```

存储过程包括过程首部和过程体。在过程首部中,“过程名”是数据库服务器合法的对象标识;存储过程提供了 IN、OUT、INOUT 三种参数模式,分别对应输入、输出、输入输出三种语义,不声明参数模式时默认为 IN 类型。输入参数在被调用时需要指定参数值,输出参数调用时不传入参数值,而是作为返回值返回。输入输出参数调用时需要传入初始值,并会返回操作后的最终值。参数列表中需要指定参数模式、参数名以及参数的数据类型。

过程体是一个<过程化 SQL 块>,包括声明部分和可执行语句部分。<过程化 SQL 块>的基本结构已经在 8.2.1 小节中讲解了。

[例 8.5] 给定学生学号, 计算学生的平均学分绩点。

```
CREATE OR REPLACE PROCEDURE compGPA( /* 定义存储过程 compGPA */
    IN inSno CHAR(10); /* 输入参数: 学生学号 inSno */
    OUT outGPA FLOAT) /* 输出参数: 平均学分绩点 outGPA */
AS
DECLARE
    courseGPA INT; /* 声明变量 courseGPA, 临时存储课程绩点 */
    totalGPA INT; /* 声明变量 totalGPA, 临时存储总学分绩点 */
    totalCredit INT; /* 声明变量 totalCredit, 临时存储总学分 */
    grade INT; /* 声明变量 grade, 临时存储学生成绩 */
    credit INT; /* 声明变量 credit, 临时存储课程学分 */
    mycursor CURSOR FOR /* 声明游标 mycursor */
    SELECT Ccredit, grade FROM SC, Course
        WHERE Sno=inSno and SC. Cno=Course. Cno;
BEGIN
    totalGPA:=0;
    totalCredit:=0;
    OPEN mycursor; /* 打开游标 mycursor */
    LOOP /* 循环遍历游标 */
        FETCH mycursor INTO credit, grade; /* 检索游标 */
        EXIT WHEN mycursor%NOTFOUND;
        IF grade BETWEEN 90 AND 100 THEN courseGPA:=4.0;
        ELSEIF grade BETWEEN 80 AND 89 THEN courseGPA:=3.0;
        ELSEIF grade BETWEEN 70 AND 72 THEN courseGPA:=2.0;
        ELSEIF grade BETWEEN 60 AND 69 THEN courseGPA:=1.0;
        ELSE courseGPA:=0;
        END IF; /* 参照表 8.2, 根据成绩找出某门课程对应的绩点 */
        totalGPA:=totalGPA+courseGPA * credit;
        totalCredit:=totalCredit+credit;
    END LOOP;
    CLOSE mycursor; /* 关闭游标 mycursor */
    outGPA:=1.0 * totalGPA/totalCredit;
END;
```

2. 执行存储过程

CALL/PERFORM [PROCEDURE] 过程名([参数 1, 参数 2, ...]);

使用 CALL 或者 PERFORM 等方式激活存储过程的执行^①。在过程化 SQL 中, 数据库服务器支持在过程体中调用其他存储过程。

[例 8.6] 查询学号为 20180001 的学生的平均学分绩点 GPA。

```
DECLARE outGPA FLOAT;  
BEGIN  
    CALL compGPA('20180001',outGPA);  
    RAISE NOTICE 'GPA: %', outGPA;  
END;
```

在调用含有输入参数和输入输出参数的存储过程时, 需要指定具体的参数值。在调用含有输出参数的存储过程时, 对应位置不需要传入参数值, 但需要事先定义输出变量, 如例 8.6 所示。

3. 修改存储过程

可以使用 ALTER PROCEDURE 重命名一个存储过程, 格式如下:

```
ALTER PROCEDURE 过程名 1 RENAME TO 过程名 2;
```

可以使用 ALTER PROCEDURE 重新编译一个存储过程, 格式如下:

```
ALTER PROCEDURE 过程名 COMPILE;
```

4. 删除存储过程

```
DROP PROCEDURE 过程名;
```

5. 存储过程的优点

使用存储过程具有以下一些优点:

① 存储过程不像解释执行的 SQL 语句那样在提出操作请求时才进行语法分析和优化工作, 而是在创建存储过程时就完成了, 因而它的运行效率更高, 它提供了在服务器端快速执行 SQL 语句的有效途径。

② 存储过程降低了客户机和服务器之间的通信量。客户机上的应用程序只要通过网络向服务器发出调用存储过程的名字和参数, 就可以让关系数据库管理系统执行其中的多条 SQL 语句并进行数据处理。只有最终的处理结果才返回客户端。

③ 方便实施企业规则。可以把企业规则的运算程序写成存储过程放入数据库服务器中, 由关系数据库管理系统管理。这样做既有利于集中控制, 又能够方便地进行维护。当企业规则发生变化时只要修改存储过程即可, 无须修改其他应用程序。

8.2.6 存储函数

存储函数也称为自定义函数, 以区别于关系数据库管理系统的内置函数。存储函数和存储

^① 有些数据库管理系统, 如 Oracle、MySQL、Kingbase 等, 其 CALL 语句不含 PROCEDURE 关键字。

过程类似，都是持久性存储模块。存储函数的创建和存储过程也类似，不同之处是存储函数必须指定返回的类型。

1. 创建存储函数

```
CREATE OR REPLACE FUNCTION 函数名([参数 1 数据类型, 参数 2 数据类型, ...])  
RETURNS <类型>  
AS <过程化 SQL 块>;
```

2. 执行存储函数

```
CALL/SELECT 函数名([参数 1, 参数 2, ...]);
```

3. 修改存储函数

可以使用 ALTER FUNCTION 重命名一个存储函数，格式如下：

```
ALTER FUNCTION 函数名 1 RENAME TO 函数名 2;
```

可以使用 ALTER FUNCTION 重新编译一个存储函数，格式如下：

```
ALTER FUNCTION 函数名 COMPILE;
```

存储函数的概念与存储过程类似，这里就不再赘述了。有关示例可参见本章参考文献[4]。

8.3 JDBC 编程

Java 是一种面向对象的高级编程语言，具有健壮性、安全性、可移植性等优点。考虑到 Java 是最受欢迎的编程语言之一，全球有超过千万的开发者在使用它，本节将介绍如何进行 JDBC 编程，即如何使用 Java 语言调用 JDBC 来进行数据库应用程序的设计。

8.3.1 JDBC 工作原理概述

JDBC 应用系统的体系结构如图 8.5 所示，它由三部分构成：用户应用程序、JDBC 驱动程序管理器和数据源。

1. 用户应用程序

用户应用程序提供用户界面、应用逻辑和事务逻辑。使用 JDBC 开发数据库应用程序时，不用关心底层数据源是什么，只需要知道交互数据源的类型即可。用户应用程序与数据库中数据进行交互时，调用的是 JDBC 提供的标准接口(JDBC APIs)。JDBC 的标准接口主要包括：驱动程序管理接口(java.sql.DriverManager)、请求数据库连接时的连接接口(java.sql.Connection)、向数据源发送 SQL 语句并接收结果的执行接口(java.sql.Statement、java.sql.PreparedStatement、java.sql.CallableStatement)、数据库操作结果集处理接口(java.sql.ResultSet)等。